

GPIO Configuration

Overview

This codebase is designed to configure and initialize General Purpose Input/Output (GPIO) pins on a Texas Instruments microcontroller (likely from the C2000 series, given the use of `driverlib.h` and `device.h`). The code sets up multiple GPIO pins as inputs for specific functions, categorized as RDY (Ready), DEN (Data Enable), and DSEL (Data Select) pins. The code is structured across four files: `gpio\_config.h`, `gpio\_setup.c`, `gpio\_setup.h`, and `main.c`.

The primary purpose is to initialize the microcontroller's GPIO pins to monitor external signals, likely for a control or communication application. The code uses the Texas Instruments DriverLib library to interface with the microcontroller's peripherals.

File Descriptions and Code

1. `gpio\_config.h`

This header file defines macros for GPIO pin numbers and their corresponding configurations for RDY, DEN, and DSEL pins.

Key Components:

Macros:

RDY Pins: Defines two pins (`RDY1` on GPIO29 and `RDY2` on GPIO12) for ready signals.

DEN Pins: Defines four pins (`DEN1` on GPIO20, `DEN2` on GPIO28, `DEN3` on GPIO24, `DEN4` on GPIO33) for data enable signals.

DSEL Pins: Defines two pins (`DSEL1` on GPIO13 and `DSEL2` on GPIO11) for data select signals.

Each pin is assigned a number (`GPIO\_NUM`) and a configuration macro (`GPIO\_PIN\_CONFIG`) that maps the pin to its GPIO function (e.g., `GPIO\_29\_GPIO29`).

Code:

```
#ifndef GPIO_GPIO_CONFIG_H_
#define GPIO_GPIO_CONFIG_H_

// ----- RDY Pins -----

#define RDY1_GPIO_NUM 29
#define RDY1_GPIO_PIN_CONFIG GPIO_29_GPIO29
#define RDY2_GPIO_NUM 12
#define RDY2_GPIO_PIN_CONFIG GPIO_12_GPIO12

// ----- DEN Pins -----

#define DEN1_GPIO_NUM 20
#define DEN1_GPIO_PIN_CONFIG GPIO_20_GPIO20
#define DEN2_GPIO_NUM 28
```

```

#define DEN2_GPIO_PIN_CONFIG GPIO_28_GPIO28
#define DEN3_GPIO_NUM 24
#define DEN3_GPIO_PIN_CONFIG GPIO_24_GPIO24
#define DEN4_GPIO_NUM 33
#define DEN4_GPIO_PIN_CONFIG GPIO_33_GPIO33
// ----- DSEL Pins -----
#define DSEL1_GPIO_NUM 13
#define DSEL1_GPIO_PIN_CONFIG GPIO_13_GPIO13
#define DSEL2_GPIO_NUM 11
#define DSEL2_GPIO_PIN_CONFIG GPIO_11_GPIO11
#endif /* GPIO_GPIO_CONFIG_H_ */

```

Notes:

The macros provide a clear mapping of GPIO pins to specific functions, making the code modular and easy to modify for different pin assignments.

-The organization into RDY, DEN, and DSEL categories suggests a specific application, such as interfacing with a peripheral or communication module.

2. `gpio\_setup.h`

This header file declares the function prototypes for GPIO initialization and input handling.

Key Components:

Function Prototypes:

`Init_gpio()`: Initializes the GPIO pins as inputs with the specified configurations.

`input_gpio()`: Declared but not implemented in the provided code, likely intended for reading or processing GPIO input states.

Code:

```

#ifndef GPIO_GPIO_SETUP_H_
#define GPIO_GPIO_SETUP_H_

// Function declarations

extern void Init_gpio(void);
extern void input_gpio(void);

#endif /* GPIO_GPIO_SETUP_H_ */

```

Notes:

The ``input_gpio()`` function is declared but not defined, indicating that it may be implemented elsewhere or intended for future use.

The header file ensures that the ``Init_gpio()`` function is accessible to other modules.

3. ``gpio_setup.c``

This source file implements the ``Init_gpio()`` function, which configures the RDY, DEN, and DSEL pins as inputs.

Key Components:**Function: `Init_gpio()`:**

Configures each pin using ``GPIO_setPinConfig()`` to map the pin to its GPIO function (e.g., ``GPIO_29_GPIO29`` for RDY1).

Sets the pad configuration to standard mode (``GPIO_PIN_TYPE_STD``) for all pins.

Sets the qualification mode to synchronous (``GPIO_QUAL_SYNC``) to ensure stable input sampling.

Configures all pins as inputs using ``GPIO_setDirectionMode(GPIO_DIR_MODE_IN)``.

Code:

```
#include "driverlib.h"
#include "device.h"
#include "gpio_config.h"

void Init_gpio(void)
{
    // RDY Pins
    GPIO_setPinConfig(RDY1_GPIO_PIN_CONFIG);
    GPIO_setPadConfig(RDY1_GPIO_NUM, GPIO_PIN_TYPE_STD);
    GPIO_setQualificationMode(RDY1_GPIO_NUM, GPIO_QUAL_SYNC);
    GPIO_setDirectionMode(RDY1_GPIO_NUM, GPIO_DIR_MODE_IN);
    GPIO_setPinConfig(RDY2_GPIO_PIN_CONFIG);
    GPIO_setPadConfig(RDY2_GPIO_NUM, GPIO_PIN_TYPE_STD);
    GPIO_setQualificationMode(RDY2_GPIO_NUM, GPIO_QUAL_SYNC);
    GPIO_setDirectionMode(RDY2_GPIO_NUM, GPIO_DIR_MODE_IN);
    // DEN Pins
    GPIO_setPinConfig(DEN1_GPIO_PIN_CONFIG);
    GPIO_setPadConfig(DEN1_GPIO_NUM, GPIO_PIN_TYPE_STD);
```

```

GPIO_setQualificationMode(DEN1_GPIO_NUM, GPIO_QUAL_SYNC);
GPIO_setDirectionMode(DEN1_GPIO_NUM, GPIO_DIR_MODE_IN);
GPIO_setPinConfig(DEN2_GPIO_PIN_CONFIG);
GPIO_setPadConfig(DEN2_GPIO_NUM, GPIO_PIN_TYPE_STD);
GPIO_setQualificationMode(DEN2_GPIO_NUM, GPIO_QUAL_SYNC);
GPIO_setDirectionMode(DEN2_GPIO_NUM, GPIO_DIR_MODE_IN);
GPIO_setPinConfig(DEN3_GPIO_PIN_CONFIG);
GPIO_setPadConfig(DEN3_GPIO_NUM, GPIO_PIN_TYPE_STD);
GPIO_setQualificationMode(DEN3_GPIO_NUM, GPIO_QUAL_SYNC);
GPIO_setDirectionMode(DEN3_GPIO_NUM, GPIO_DIR_MODE_IN);
GPIO_setPinConfig(DEN4_GPIO_PIN_CONFIG);
GPIO_setPadConfig(DEN4_GPIO_NUM, GPIO_PIN_TYPE_STD);
GPIO_setQualificationMode(DEN4_GPIO_NUM, GPIO_QUAL_SYNC);
GPIO_setDirectionMode(DEN4_GPIO_NUM, GPIO_DIR_MODE_IN);
// DSEL Pins
GPIO_setPinConfig(DSEL1_GPIO_PIN_CONFIG);
GPIO_setPadConfig(DSEL1_GPIO_NUM, GPIO_PIN_TYPE_STD);
GPIO_setQualificationMode(DSEL1_GPIO_NUM, GPIO_QUAL_SYNC);
GPIO_setDirectionMode(DSEL1_GPIO_NUM, GPIO_DIR_MODE_IN);
GPIO_setPinConfig(DSEL2_GPIO_PIN_CONFIG);
GPIO_setPadConfig(DSEL2_GPIO_NUM, GPIO_PIN_TYPE_STD);
GPIO_setQualificationMode(DSEL2_GPIO_NUM, GPIO_QUAL_SYNC);
GPIO_setDirectionMode(DSEL2_GPIO_NUM, GPIO_DIR_MODE_IN);
}

```

Notes:

The function configures eight GPIO pins consistently as inputs, ensuring they are ready to monitor external signals.

The use of synchronous qualification (`GPIO\_QUAL\_SYNC`) avoids glitches in input sampling, which is critical for reliable signal detection.

4. `main.c`

This source file contains the main program logic for initializing the microcontroller and GPIO pins, and includes a basic example of reading a GPIO pin.

Key Components:

Main Function:

Initializes the device clock and peripherals (`Device\_init()`).

Initializes the Peripheral Interrupt Expansion (PIE) module and vector table (`Interrupt\_initModule()`, `Interrupt\_initVectorTable()`).

Calls `Board\_init()` to initialize board-specific configurations (implementation not shown but assumed to be in `board.h`).

Calls `Init\_gpio()` to configure the GPIO pins.

Enables global interrupts (`Interrupt\_enableGlobal()`).

Enters an infinite loop that reads the state of the `RDY1` pin (`GPIO\_readPin(RDY1\_GPIO\_NUM)`) as an example, with a placeholder for additional logic.

Code:

```
#include "driverlib.h"

#include "device.h"

#include "board.h"

#include "gpio_config.h"

#include "gpio_setup.h"

//

// Main

//

void main(void)

{

    //

    // Initializes system control, device clock, and peripherals

    //

    Device_init();

    //

    // Initializes PIE and clear PIE registers. Disables CPU interrupts.

    // and clear all CPU interrupt flags.
```

```

//
Interrupt_initModule();
//
// Initialize the PIE vector table with pointers to the shell interrupt
// Service Routines (ISR).
//
Interrupt_initVectorTable();
//
// Board Initialization
//
Board_init();
Init_gpio();
//
// Enables CPU interrupts
//
Interrupt_enableGlobal();
//
// Loop.
//
while(1)
{
    // Example: read RDY1 pin
    uint16_t rdy1 = GPIO_readPin(RDY1_GPIO_NUM);
    // Logic depending on input
}
}

```

Notes:

The main loop includes a basic example of reading the `RDY1` pin, but no specific logic is implemented, suggesting that the code is a template or starting point.

The enabling of global interrupts suggests that interrupt-driven GPIO handling (e.g., for edge detection) may be implemented elsewhere.

Working of the Code

Initialization

1. Device Setup:

The ``main()`` function initializes the microcontroller's clock, peripherals, and interrupt system using ``Device_init()``, ``Interrupt_initModule()``, and ``Interrupt_initVectorTable()``.

``Board_init()`` (not shown) likely configures board-specific settings, such as pin multiplexing or additional peripherals.

2. GPIO Configuration:

The ``Init_gpio()`` function configures eight GPIO pins (two RDY, four DEN, and two DSEL) as inputs.

Each pin is set to standard mode with synchronous qualification and configured for input direction, ensuring reliable signal monitoring.

3. Runtime Behavior:

After initialization, the program enters an infinite loop in ``main()``.

The loop reads the state of the ``RDY1`` pin (``GPIO_readPin(RDY1_GPIO_NUM)``) as an example of input monitoring.

The placeholder comment ("Logic depending on input") indicates that additional logic can be added to process the states of RDY, DEN, and DSEL pins.

Intended Application

The code is likely intended for a control or communication system where the microcontroller monitors external signals via the RDY, DEN, and DSEL pins. Possible applications include:

Communication Interfaces: The RDY and DEN pins suggest a protocol where "Ready" and "Data Enable" signals control data transfer (e.g., SPI, I2C, or a custom protocol).

Data Selection: The DSEL pins may be used to select specific data channels or modes in a multi-channel system.

Control Systems: The pins could monitor status signals from external devices, such as sensors or other microcontrollers.

The use of multiple DEN and DSEL pins suggests a system with multiple data channels or devices, requiring coordinated input monitoring.

Potential Issues and Improvements

1. Unimplemented ``input_gpio()`` Function:

The ``input_gpio()`` function is declared in ``gpio_setup.h`` but not implemented. If intended for reading or processing GPIO inputs, it should be defined or removed.

2. Placeholder Logic in ``main()``:

The main loop only reads the ``RDY1`` pin without processing other pins or implementing specific logic. Consider adding logic to monitor all pins and handle their states based on the application's requirements.

3. Interrupt Handling:

Global interrupts are enabled, but no specific GPIO interrupts are configured. If the application requires real-time response to pin changes (e.g., edge detection), configure GPIO interrupts using ``GPIO_setInterruptPin()`` and ``Interrupt_register()``.

4. Pin Configuration Verification:

Ensure that the assigned GPIO pins (e.g., GPIO29, GPIO12, etc.) are available and not multiplexed with other functions on the target microcontroller.

Verify that the pin assignments align with the hardware schematic.

5. Documentation:

Add comments in ``gpio_setup.c`` to explain the purpose of RDY, DEN, and DSEL pins in the context of the application.

Document the expected behavior of the ``input_gpio()`` function if it is implemented later.

6. Scalability:

The GPIO configuration is hardcoded for eight pins. Consider parameterizing the ``Init_gpio()`` function to support additional pins or dynamic configuration.

Example Usage

To use this code:

1. Ensure the Texas Instruments DriverLib and device-specific headers (``driverlib.h``, ``device.h``, ``board.h``) are included in the project.
2. Implement the ``Board_init()`` function or include the appropriate board support file for the target microcontroller.
3. If interrupt-driven GPIO handling is needed, configure GPIO interrupts for the RDY, DEN, or DSEL pins and implement corresponding ISRs.
4. Add logic in the ``main()`` loop to process the states of all configured pins (e.g., reading DEN and DSEL pins in addition to ``RDY1``).
5. If the ``input_gpio()`` function is needed, implement it to handle GPIO input processing.
6. Compile and flash the code onto a compatible Texas Instruments microcontroller.

Conclusion

This codebase provides a foundation for configuring and monitoring GPIO pins on a Texas Instruments microcontroller. The structured approach (header files for configuration and function declarations, a dedicated GPIO setup file, and a main loop) makes it easy to extend for specific applications. With minor enhancements, such as implementing the ``input_gpio()`` function, adding interrupt handling, and processing all pin states, the code can support robust control or communication systems. The clear categorization of pins (RDY, DEN, DSEL) suggests a well-defined application context, likely involving data transfer or device status monitoring.